Bhumi3DMapper

QGIS Plugin — Architecture, Development Plan, Testing Plan & Deployment Prompt

Version: 1.0 | **Date:** March 2026 | **Target:** QGIS 3.28+ (Qt5) and QGIS 4.0+ (Qt6, dual-compatible)

> This document is the complete blueprint for developing, testing and deploying the Bhumi3DMapper plugin. It is structured to serve as both a human planning reference and a direct input prompt for AI-assisted development. All code references are to the ProspectivityMapper_Framework already built and available in this project.

1. Plugin Architecture

1.1 Design Principles

- The core computation modules (m01–m06, config.py, pipeline.py) NEVER import QGIS. They remain pure numpy/pandas, usable standalone and testable without QGIS.
- The QGIS plugin is a thin wrapper — it handles file picking, layer loading, progress display and result visualisation, but delegates ALL computation to the core modules.
- Qt imports always use "from qgis.PyQt import ..." — never "from PyQt5 import ..." — ensuring dual Qt5/Qt6 compatibility across QGIS 3.28 LTR, 3.44 LTR and 4.0.
- Every pipeline step is implemented as a QgsProcessingAlgorithm. This gives free batch processing, model designer integration, and a consistent UI.
- Heavy computation (scoring, voxel build) runs in QgsTask background threads. The QGIS UI never freezes.

1.2 Four-Layer Architecture

Layer	Files	Responsibility
LAYER 1 QGIS UI	bhumi3dmapper/ __init__.py metadata.txt provider.py ui/dock_panel.py ui/resources.qrc	Entry point. Registers algorithms in Processing Toolbox. Adds Bhumi3DMapper menu item under Plugins → Bhumi3DMapper. Dockable panel for project config. Qt6/Qt5 dual-compatible (qgis.PyQt imports only).
LAYER 2 Processing Algorithms	algorithms/ alg_load_data.py alg_run_scoring.py alg_gpkg_export.py alg_voxel_build.py alg_load_results.py	One QgsProcessingAlgorithm per pipeline step. Delegates all computation to core modules. Uses QgsTask background threads — no UI freeze. Progress bars, cancellation, and QGIS log integration built-in.
LAYER 3 Core Framework	../prospectivity_framework/ core/config.py modules/m01–m06 pipeline.py	NEVER imports QGIS. Pure numpy/pandas. Identical to standalone CLI. Used by algorithms via import. Testable with pytest without launching QGIS.
LAYER 4 Outputs	GeoPackage (.gpkg) per mRL level Voxel .npz archives project_config.json	2D GPKGs loaded directly into QGIS project with pre-applied symbology. Voxel on-demand. Config JSON re-importable for next session.

1.3 Menu Integration

When installed, Bhumi3DMapper appears in two locations in QGIS:

- Plugins menu → Bhumi3DMapper → Open Project Panel (opens the dockable config panel)
- Processing Toolbox → Bhumi3DMapper → five algorithm entries: Load & Validate Data, Run Prospectivity Scoring, Export GeoPackages, Build 3D Voxel, Load Existing Results

> The menu item is registered in bhumi3dmapper.py initGui() using iface.addPluginToMenu('Bhumi3DMapper', action). The Processing Toolbox registration happens via the QgsProcessingProviderRegistry in provider.py. Both registrations are reversed in unload() for clean removal.

1.4 Complete File Structure

Path	Purpose
bhumi3dmapper/	Root plugin directory — lives in QGIS plugins folder
__init__.py	Plugin entry point. classFactory() returns BhumiPlugin instance.
metadata.txt	Required: name, version, qgisMinimumVersion, author, description, tags
bhumi3dmapper.py	Main plugin class: initGui(), unload(), menu registration
provider.py	QgsProcessingProvider: registers all algorithm classes
icon.png	48×48px plugin icon (shown in toolbar + menu)
resources.qrc / resources_rc.py	Qt resource file for icon + UI assets
algorithms/	One file per pipeline step
__init__.py	
alg_load_data.py	QgsProcessingAlgorithm: validate + load all input files
alg_run_scoring.py	QgsProcessingAlgorithm: run proximity + blind models per level
alg_gpkg_export.py	QgsProcessingAlgorithm: write GeoPackages, load into project
alg_voxel_build.py	QgsProcessingAlgorithm: build full 3D voxel (background task)
alg_load_results.py	QgsProcessingAlgorithm: load existing GPKGs with symbology
ui/	Qt dock panel widgets
__init__.py	
dock_panel.py	BhumiDockWidget(QDockWidget): project config UI
config_widget.py	Tab widget: Project / Data / Geology / Structure / Scoring / Run
core/ (symlink or copy)	../prospectivity_framework/core/config.py
modules/ (symlink or copy)	../prospectivity_framework/modules/m01–m06
test/	Pytest suite
test_config.py	Config roundtrip JSON tests
test_scoring.py	Criterion function unit tests (no QGIS needed)
test_gpkg.py	GeoPackage write/read integration tests
test_algorithms.py	QgsProcessingAlgorithm headless tests (pytest-qgis)

2. Development Plan

2.1 Prerequisites

- QGIS 3.28 or later installed (OSGeo4W on Windows, or native package on Linux/Mac)
- Python 3.8+ with numpy, pandas, Pillow, scipy (already available in QGIS Python environment)
- Plugin Builder plugin installed in QGIS (for boilerplate generation, optional)
- Plugin Reloader plugin installed in QGIS (for iterative development without restart)
- Git repository initialised at the bhumi3dmapper/ directory
- ProspectivityMapper_Framework/ available at a known path (either co-located or referenced by absolute path in config)

2.2 Sprint-by-Sprint Development Plan (10 weeks)

#	Phase	Work items	Done when	Risk
1	Sprint 1 Week 1-2	Plugin skeleton: __init__.py, metadata.txt, bhumi3dmapper.py, provider.py, icon.png. Register empty provider. Verify 'Bhumi3DMapper' appears in Plugins menu and Processing Toolbox after install.	Plugin visible in QGIS menu and Toolbox	Low
2	Sprint 2 Week 3	Dock panel UI: config_widget.py with 6 tabs (Project, Data, Geology, Structure, Scoring, Run). All field types: file pickers, number inputs, dropdowns. Export project_config.json button.	Config UI renders; JSON exports correctly	Medium
3	Sprint 3 Week 4	alg_load_data.py: QgsProcessingAlgorithm wrapping DataLoader.validate_all(). Parameter types: QgsProcessingParameterFile for each input. Test with Kayad CSVs and TIF folders.	Validation passes/fails correctly with test data	Medium
4	Sprint 4 Week 5-6	alg_run_scoring.py: wraps pipeline steps drill+geophys+scoring via QgsTask. Progress feedback to QGIS log. Cancellation support. Run for 3 test levels (185, 235, 310 mRL).	Score output matches standalone pipeline to 3 decimal places	High
5	Sprint 5 Week 7	alg_gpkg_export.py: write per-level GPKGs, auto-load into QGIS project with graduated symbology on prox_score field, Spectral colour ramp, 5 classes. Layer names: 'Proximity mRL+185' etc.	GPKGs load with correct symbology; spatial index confirmed	High
6	Sprint 6 Week 8	alg_voxel_build.py: full 145-level voxel via QgsTask background thread. Progress bar 0-100%. Non-blocking UI during build. alg_load_results.py: load existing GPKG stack.	Voxel builds without freezing QGIS; all 145 levels correct	High
7	Sprint 7 Week 9	Qt6 dual-compatibility pass: replace all PyQt5 imports with qgis.PyQt. Run pyqt5_to_pyqt6.py migration checker. Test on QGIS 3.28 LTR, 3.44, and 4.0.	Zero Qt5-specific imports; passes on all 3 QGIS versions	Critical
8	Sprint 8 Week 10	Packaging: build ZIP installer, test install from Plugins → Install from ZIP. Write install guide. Submit to QGIS Plugin Repository (optional).	One-click install works on clean QGIS 3.28+ and 4.0	High

2.3 Key Implementation Details

2.3.1 metadata.txt (required by QGIS)

```
[general]
name=Bhumi3DMapper
qgisMinimumVersion=3.28
qgisMaximumVersion=4.99
description=3D mineral prospectivity mapping for any deposit type
version=1.0.0
author=AiRE — AI Resource Exploration
email=your@email.com
about=Config-driven multi-criterion prospectivity pipeline. Proximity and Blind
      environment models. SEDEX Pb-Zn, VMS, porphyry, epithermal and more.
tracker=https://github.com/yourorg/bhumi3dmapper/issues
repository=https://github.com/yourorg/bhumi3dmapper
tags=geology,mining,prospectivity,geophysics,3D,voxel
homepage=https://github.com/yourorg/bhumi3dmapper
category=Analysis
icon=icon.png
experimental=False
deprecated=False
```

2.3.2 __init__.py (QGIS entry point)

```
def classFactory(iface):
    from .bhumi3dmapper import Bhumi3DMapper
    return Bhumi3DMapper(iface)
```

2.3.3 bhumi3dmapper.py — Menu registration pattern

```
from qgis.PyQt.QtWidgets import QAction, QDockWidget
from qgis.PyQt.QtGui import QIcon
import os

class Bhumi3DMapper:
    def __init__(self, iface):
        self.iface = iface
        self.plugin_dir = os.path.dirname(__file__)
        self.actions = []
        self.menu = 'Bhumi3DMapper'
        self.dock = None

    def initGui(self):
        # Register Processing provider
        from .provider import Bhumi3DProvider
        self.provider = Bhumi3DProvider()
        QgsApplication.processingRegistry().addProvider(self.provider)
        # Add menu item
        icon = QIcon(os.path.join(self.plugin_dir, 'icon.png'))
        action = QAction(icon, 'Open Project Panel', self.iface.mainWindow())
        action.triggered.connect(self.openPanel)
        self.iface.addPluginToMenu(self.menu, action)
        self.iface.addToolBarIcon(action)
        self.actions.append(action)

    def unload(self):
        QgsApplication.processingRegistry().removeProvider(self.provider)
        for action in self.actions:
            self.iface.removePluginMenu(self.menu, action)
            self.iface.removeToolBarIcon(action)
        if self.dock:
            self.iface.removeDockWidget(self.dock)

    def openPanel(self):
        from .ui.dock_panel import BhumiDockWidget
```

```
        if not self.dock:
            self.dock = BhumiDockWidget(self.iface)
            self.iface.addDockWidget(Qt.RightDockWidgetArea, self.dock)
        self.dock.show()
```

2.3.4 provider.py — Algorithm registration

```
from qgis.core import QgsProcessingProvider
from qgis.PyQt.QtGui import QIcon
import os

class Bhumi3DProvider(QgsProcessingProvider):
    def loadAlgorithms(self):
        from .algorithms.alg_load_data import LoadDataAlgorithm
        from .algorithms.alg_run_scoring import RunScoringAlgorithm
        from .algorithms.alg_gpkg_export import GpkgExportAlgorithm
        from .algorithms.alg_voxel_build import VoxelBuildAlgorithm
        from .algorithms.alg_load_results import LoadResultsAlgorithm
        for alg in [LoadDataAlgorithm, RunScoringAlgorithm,
                    GpkgExportAlgorithm, VoxelBuildAlgorithm, LoadResultsAlgorithm]:
            self.addAlgorithm(alg())

    def id(self):    return 'bhumi3dmapper'
    def name(self):  return 'Bhumi3DMapper'
    def icon(self):  return QIcon(os.path.join(os.path.dirname(__file__), 'icon.png'))
    def longName(self): return '3D Mineral Prospectivity Mapper'
```

2.3.5 Auto-load result GPKGs with symbology

```
from qgis.core import QgsVectorLayer, QgsProject, QgsGraduatedSymbolRenderer
from qgis.PyQt.QtGui import QColor

def load_gpkg_with_symbology(gpkg_path, z_mrl, model='prox'):
    score_field = 'prox_score' if model == 'prox' else 'blind_score'
    layer_name  = f'{'Proximity' if model=='prox' else 'Blind'} mRL{int(z_mrl):+04d}'
    layer = QgsVectorLayer(gpkg_path, layer_name, 'ogr')
    if not layer.isValid(): return None
    # Apply Spectral graduated symbology on score field
    renderer = QgsGraduatedSymbolRenderer.createRenderer(
        layer, score_field, 5,
        QgsGraduatedSymbolRenderer.Pretty,
        QgsStyle.defaultStyle().colorRamp('Spectral'))
    renderer.setInverted(True)   # Red = high score
    layer.setRenderer(renderer)
    QgsProject.instance().addMapLayer(layer)
    return layer
```

3. Testing Plan

3.1 Test Philosophy

The testing pyramid for Bhumi3DMapper has three levels. The base — unit tests — runs in milliseconds with no QGIS dependency. The middle — integration tests — tests file I/O and GPKG validity. The apex — system tests — tests the full QGIS plugin manually on three target versions.

- Unit tests cover all criterion functions in m04_scoring_engine.py with boundary conditions. These are the most important tests because they verify the geological logic.
- Integration tests verify that the full pipeline produces valid, correctly structured GeoPackage files.
- System tests verify that the plugin installs, appears in menus, and produces correct visual output in real QGIS sessions.

3.2 Test Suite

Type	Test suite / method	What is tested	How to run	Pass criterion
Unit	**test_config.py**	ProjectConfig serialises to JSON and deserialises back identically. All 10 scoring criteria return values in [0,1]. Hard veto caps score at 20.	pytest test/ -v (no QGIS)	Pass/Fail
Unit	**test_scoring.py**	Each criterion function: input boundary values (0m, 200m, 999m), regime transitions (mRL 60, 160), amphibolite veto. Both proximity and blind models produce scores in [0,100].	pytest test/test_scoring.py	All asserts pass
Integration	**test_gpkg.py**	Write a 5-level GPKG stack with synthetic data. Verify: EPSG correct, all 36 fields present, spatial index exists, score range valid, prox_class_name populated.	pytest test/test_gpkg.py	File valid, spatial query returns results
Integration	**test_algorithms.py**	QgsProcessingAlgorithm headless run via pytest-qgis. Load Kayad collar/litho/gravity for 3 levels. Verify output GPKGs created, layer added to project.	pytest-qgis test/test_algorithms.py	GPKG files exist; layers in project
System	**Manual QGIS 3.28**	Install from ZIP. Verify menu item appears under Plugins → Bhumi3DMapper → Open Panel. Dock panel opens. Run full pipeline on Kayad data mRL 160-235. Check symbology auto-applied.	Manual + screenshot log	Menu present; GPKGs load with colour ramp
System	**Manual QGIS 3.44**	Same as above on LTR release. Verify no deprecation warnings in QGIS log panel.	Manual	Zero warnings
System	**Manual QGIS 4.0**	Same as above on Qt6 release. Run pyqgis4-checker. Zero Qt5-specific import errors.	pyqgis4-checker + manual	Zero Qt5 import errors
Performance	**Voxel build timing**	Full 145-level voxel build completes in under 15 minutes on a standard workstation	Timer in alg_voxel_build.py	<15 min; no UI freeze

		(16GB RAM, SSD). UI remains responsive throughout (QgsTask).		
Regression	**CI pipeline**	On every git push: pytest unit tests run headlessly. pyqt5_to_pyqt6.py linter runs. Both must pass before merge.	GitHub Actions / GitLab CI	Green badge
Deposit adaptation	**VMS synthetic data**	Change rock_codes to felsic volcanic / mafic. Remove PG halo criterion (weight=0). Run on synthetic VMS dataset. Verify output reflects new deposit model.	pytest + manual	New lithology codes score correctly

3.3 CI Configuration (GitHub Actions)

```
# .github/workflows/test.yml
name: Bhumi3DMapper Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: pip install numpy pandas Pillow scipy pytest
      - name: Run unit tests (no QGIS needed)
        run: pytest bhumi3dmapper/test/test_config.py bhumi3dmapper/test/test_scoring.py
-v
      - name: Qt6 compatibility check
        run: |
          docker run --rm -v $(pwd):/code \
            registry.gitlab.com/oslandia/qgis/pyqgis-4-checker/pyqgis-qt-checker:latest \
            pyqt5_to_pyqt6.py --dry_run /code/bhumi3dmapper/
```

3.4 Manual System Test Checklist

Run this checklist on each target QGIS version before each release:

1. Install plugin from ZIP via Plugins → Manage and Install Plugins → Install from ZIP
2. Verify 'Bhumi3DMapper' appears in the Plugins menu
3. Verify 'Bhumi3DMapper' appears in Processing Toolbox with 5 sub-algorithms
4. Click Plugins → Bhumi3DMapper → Open Project Panel. Confirm dock opens on right side.
5. Fill in Project tab: name, deposit type (SEDEX Pb-Zn), CRS 32643
6. Fill in Data tab: point to Kayad collar/assay/litho CSVs, gravity TIF folder, mag TIF folder
7. Click 'Run → Load & Validate Data'. Confirm green status in QGIS log panel.
8. Click 'Run → Run Scoring' with levels 185, 235, 310. Watch progress bar.
9. After run: confirm 3 new layers appear in QGIS with names 'Proximity mRL+185' etc.
10. Confirm layers use Spectral graduated colour ramp with 5 classes.
11. Open Identify tool. Click a red (Very High) cell. Confirm all 36 attributes visible.
12. Filter layer: prox_score >= 75. Confirm only VH cells visible.
13. Unload plugin via Plugins → Manage → Disable Bhumi3DMapper. Confirm menu item disappears.
14. Re-enable. Confirm all tools return.
15. Check QGIS log panel: zero deprecation warnings, zero Qt5-specific import errors.

4. Deployment

4.1 Building the Installable ZIP

```
# From the parent directory containing bhumi3dmapper/
# Step 1: compile Qt resources
pyrcc5 bhumi3dmapper/resources.qrc -o bhumi3dmapper/resources_rc.py
# OR for Qt6:
pyrcc6 bhumi3dmapper/resources.qrc -o bhumi3dmapper/resources_rc.py

# Step 2: create ZIP (exclude test/ and __pycache__/)
zip -r Bhumi3DMapper_v1.0.zip bhumi3dmapper/ \
    --exclude '*/test/*' \
    --exclude '*/__pycache__/*' \
    --exclude '*.pyc' \
    --exclude '*.git*'
```

4.2 Installation in QGIS

- Open QGIS → Plugins → Manage and Install Plugins → Install from ZIP
- Browse to Bhumi3DMapper_v1.0.zip → Install Plugin
- Go to Plugins → Installed → ensure Bhumi3DMapper is checked (enabled)
- Plugins menu will now show: Plugins → Bhumi3DMapper → Open Project Panel
- Processing Toolbox will show: Bhumi3DMapper → 5 algorithms

4.3 Manual Install (developer mode)

```
# Windows
copy bhumi3dmapper/
C:\Users\[username]\AppData\Roaming\QGIS\QGIS3\profiles\default\python\plugins\

# Linux / Mac
cp -r bhumi3dmapper/ ~/.local/share/QGIS/QGIS3/profiles/default/python/plugins/

# Then in QGIS: Plugins → Plugin Reloader → select bhumi3dmapper → Reload
```

4.4 QGIS Plugin Repository Submission (optional)

- Ensure metadata.txt is complete with tracker, repository, homepage URLs
- Create account at https://plugins.qgis.org
- Upload ZIP. Plugin will appear under 'Analysis' category.
- Mark as experimental=True for first public release. Set False after system test checklist passes.

5. Detailed Development Prompt

The following is a complete, self-contained prompt for AI-assisted development of Bhumi3DMapper. Paste this into the Bhumi3DMapper Claude Project to begin any development session. It references files already in the project knowledge base.

> USAGE: Copy this entire prompt into a new conversation inside the Bhumi3DMapper Claude Project. The project already contains all code modules (m01-m06, config.py, pipeline.py) and the geological report. This prompt gives the AI assistant the full context needed to write any component of the plugin from scratch without further explanation.

START OF DEVELOPMENT PROMPT

```
You are building the Bhumi3DMapper QGIS plugin. The core computation framework (9 Python files) is
already complete and available in this project. Your task is to build the thin QGIS wrapper layer that
makes it accessible as a plugin.

═══════════════════════════════════════
CORE FRAMEWORK (already built — do not modify)
═══════════════════════════════════════
- core/config.py: ProjectConfig dataclass, all project variables, JSON serialisation
- modules/m01_data_loader.py: loads CSV drill data, TIF geophysics, GPKG polygons
- modules/m02_drill_processor.py: builds litho/PG-contact/CSR spatial lookups
- modules/m03_geophys_processor.py: gravity gradient, Laplacian, level interpolation
- modules/m04_scoring_engine.py: all criterion scoring functions, both models
- modules/m05_gpkg_writer.py: writes 2D GeoPackage per level
- modules/m06_voxel_builder.py: full 3D voxel builder
- pipeline.py: standalone CLI orchestrator

═══════════════════════════════════════
WHAT YOU ARE BUILDING: the QGIS plugin wrapper
═══════════════════════════════════════
Plugin directory structure:
bhumi3dmapper/
  __init__.py
  metadata.txt
  bhumi3dmapper.py          ← main plugin class, menu registration
  provider.py               ← QgsProcessingProvider, registers algorithms
  icon.png                  ← 48x48 plugin icon
  resources.qrc             ← Qt resource file
  resources_rc.py           ← compiled resources (pyrcc5/pyrcc6)
  algorithms/
    __init__.py
    alg_load_data.py        ← wraps DataLoader.validate_all()
    alg_run_scoring.py      ← wraps full scoring pipeline via QgsTask
    alg_gpkg_export.py      ← writes GPKGs, loads into project with symbology
    alg_voxel_build.py      ← builds voxel in QgsTask background thread
    alg_load_results.py     ← loads existing GPKGs with colour ramp
  ui/
    __init__.py
    dock_panel.py           ← BhumiDockWidget(QDockWidget)
    config_widget.py        ← 6-tab config UI

═══════════════════════════════════════
ABSOLUTE REQUIREMENTS
═══════════════════════════════════════
1. ALL Qt imports MUST use: from qgis.PyQt import ...
   NEVER use: from PyQt5 import ... or from PyQt6 import ...
   This is mandatory for QGIS 3.28 AND 4.0 compatibility.

2. QGIS minimum version: 3.28 (set in metadata.txt qgisMinimumVersion)

3. Menu must appear at: Plugins → Bhumi3DMapper → Open Project Panel

4. Processing Toolbox must show: Bhumi3DMapper → [5 algorithms]

5. Core modules (m01-m06, config.py) must NEVER be modified.
   The plugin imports them, it does not change them.
```

```
6. Heavy computation (scoring, voxel) MUST use QgsTask:
   task = QgsTask.fromFunction('Bhumi3DMapper: Scoring', run_fn, on_finished=finished_fn)
   QgsApplication.taskManager().addTask(task)
   This prevents QGIS UI from freezing.

7. After scoring, GPKGs must auto-load into QGIS with:
   - Graduated symbology on prox_score field
   - Spectral colour ramp, inverted (red=high)
   - 5 classes matching: Very Low / Low / Moderate / High / Very High
   - Layer names: "Proximity mRL+185", "Blind mRL+185" etc.
```

WHAT THE DOCK PANEL MUST CONTAIN

Tab 1 — Project: project name, deposit type dropdown (SEDEX Pb-Zn / VMS Cu-Zn / Epithermal Au / Porphyry / Orogenic Au / Skarn / Custom), location text, CRS EPSG number, output directory, grid cell size, Z top mRL, Z bottom mRL, Z step.

Tab 2 — Data: file pickers for collar CSV, assay CSV, litho CSV, survey CSV; folder pickers for gravity TIF folder, magnetics TIF folder, ore polygon folder, block model folder; ore threshold % spinbox.

Tab 3 — Geology: table of rock codes → role (Primary host / Secondary host / Hard veto / Structural marker / Footwall horizon / Other); depth regime boundaries (upper/transition/lower mRL thresholds).

Tab 4 — Structure: two corridor blocks (editable), each with: name, azimuth, plunge, plunge azimuth, anchor E/N/mRL, lateral shift E and N per 100m, active mRL range.

Tab 5 — Scoring: proximity model weights table (9 criteria), blind model weights table (10 criteria), score threshold spinboxes (Very High >=75, High >=60), novelty distance spinbox.

Tab 6 — Run: step checkboxes (Load Data / Run Scoring / Export GPKGs / Build Voxel / Load Results), mRL levels text input (e.g. 185,210,235 or "all"), Generate Command button showing CLI equivalent, Run in QGIS button executing via QgsTask.

PROJECT CONFIG JSON INTEGRATION

The dock panel reads from and writes to ProjectConfig (core/config.py).
- On panel open: load last used config from QSettings or a saved JSON file
- Export button: calls cfg.to_json(path) to save current settings
- Import button: calls ProjectConfig.from_json(path) to load settings
- Run button: builds a ProjectConfig from current UI state before running

ALGORITHM PATTERN (use for all 5 algorithms)

```
from qgis.core import (QgsProcessingAlgorithm, QgsProcessingParameterFile,
                       QgsProcessingParameterString, QgsProcessingOutputString,
                       QgsProcessingContext, QgsProcessingFeedback)

class LoadDataAlgorithm(QgsProcessingAlgorithm):
    CONFIG = 'CONFIG'

    def initAlgorithm(self, config=None):
        self.addParameter(QgsProcessingParameterFile(
            self.CONFIG, 'Project config JSON',
            QgsProcessingParameterFile.File, 'json'))

    def processAlgorithm(self, parameters, context, feedback):
        config_path = self.parameterAsFile(parameters, self.CONFIG, context)
        from ..core.config import ProjectConfig
        from ..modules.m01_data_loader import DataLoader
        cfg = ProjectConfig.from_json(config_path)
        loader = DataLoader(cfg)
        ok = loader.validate_all()
        feedback.pushInfo(f'Validation: {"PASSED" if ok else "FAILED"}')
        if feedback.isCanceled(): return {}
        return {'RESULT': 'ok' if ok else 'failed'}

    def name(self):         return 'loaddata'
```

```
    def displayName(self):  return 'Load & Validate Data'
    def group(self):        return 'Bhumi3DMapper'
    def groupId(self):      return 'bhumi3dmapper'
    def createInstance(self): return LoadDataAlgorithm()
```

```
═══════════════════════════════════════
KAYAD REFERENCE DATA (for testing)
═══════════════════════════════════════
When testing, use these known-good values from the Kayad project:
- CRS: EPSG:32643 (WGS84/UTM Zone 43N)
- Grid: xmin=468655, ymin=2932890, nx=482, ny=722, cell=5m
- Z range: mRL -260 to +460 at 5m steps (145 levels)
- Primary ore centroid: E469,519 / N2,934,895 at mRL 185
- Peak proximity score: 90.6 at mRL 310
- Peak blind score: 93.0 at mRL 435 (NE corridor, E470,148/N2,936,252)
- N28E corridor: anchor E469519/N2934895/mRL185, plunge 30/075E, shift 1.22E/0.33N per 100m
- N315E deep corridor: anchor E470210/N2935041/mRL-140, plunge 12/063E, shift 4.10E/1.71N per 100m

═══════════════════════════════════════
START WITH THIS TASK
═══════════════════════════════════════
Build the plugin skeleton first (Sprint 1):
1. Create metadata.txt with all required fields
2. Create __init__.py with classFactory
3. Create bhumi3dmapper.py with initGui/unload/openPanel
4. Create provider.py registering 5 empty algorithm stubs
5. Verify it loads in QGIS showing the menu item

Then proceed to Sprint 2 (dock panel), Sprint 3 (data loader algorithm), etc.

For each sprint, run the test suite and the manual checklist before proceeding.
```

END OF DEVELOPMENT PROMPT

— END OF DOCUMENT —

Bhumi3DMapper Plugin — Architecture, Development & Testing Plan | March 2026